PES UNIVERSITY

SOFTWARE ARCHITECTURE DOCUMENT

Team 31 – Fantastic Four

Problem Statement 39

Dehallucination Add-On Module for Enhancing Reliability of AI Applications

Jackfruit Phase-1 Mini Project

Team Details	Information
Team Name	FANTASTIC FOUR
Team Number	31
Project ID (per SRS)	31
Problem Statement	39
Reference	SRS Version 1.0, 1 September 2026
Document Status	Final design document; implementation has not started

Team Member	SRN
Kushal J	PES2UG24AM205
Laxmikant	PES2UG24AM206
Gagan K C	PES2UG24AM201
Yashwanth G	PES2UG24AM189

Version	Date	Description	Author
1.0	24 October 2026	Final architecture based on SRS v1.0, the approved technology stack and the approved diagram set	Project Team

1. Introduction

1.1 Purpose

This document describes the software architecture of the Dehallucination Add-On Module. It explains how the system is divided into layers and components, how they communicate, what data is stored, which technologies are used, and why the main design decisions were made. It is the design input for the Jira Backlog and for the implementation.

1.2 Scope

The architecture covers all features of SRS v1.0 (Sections 5.1 to 5.10) and the nonfunctional requirements in Section 6. It is written before coding starts, so technology choices are marked as proposed and may be refined during implementation without changing the layer and component structure. This is a design document: it does not claim that any part of the software has been implemented or tested.

1.3 References

- Software Requirements Specification, Problem Statement 39, Version 1.0 (1 September 2026).
- PES University JACKFRUIT SE Mini-Project Guidelines 2026.

1.4 Definitions

Terms such as claim, evidence, trusted source, hallucination and confidence score are used as defined in SRS Appendix A. NLI means Natural Language Inference: a model that decides whether a piece of text (the premise) supports, contradicts, or is neutral to another statement (the hypothesis). The three official claim statuses are Supported, Contradicted and Uncertain (SRS FR-13, Appendix B).

2. Architectural Goals and Constraints

ID	Goal / Constraint	Source in SRS	How the architecture responds
AG-1	Work as an add-on; the underlying AI model is not replaced	1.3, 2.5	The module exposes a REST API and only receives generated text
AG-2	Separable, independently testable components	6.4	Layered design; each pipeline step is its own module with a defined interface
AG-3	Never present unverified content as verified	2.5, 6.2, FR-15	Fail-safe rule: missing evidence, failures and neutral NLI results map to Uncertain
AG-4	Only configured trusted sources are used	6.5	Sources come from the database through the Trusted Source Manager (C-11); none are hard-coded
AG-5	Separate generated content from evidence	2.5	The result keeps the original text, the claim results and the evidence as different items
AG-6	Protect admin functions and data	2.5, 6.3, FR-29, FR-30	Authentication and role-based access control (C-03) in the API layer
AG-7	Acceptable response time and graceful degradation	6.1	Parallel retrieval, per-source timeouts, bounded retries, caching, models loaded once at start-up
AG-8	Configurable parameters kept outside core logic	7	Configuration Manager (C-12) for thresholds, timeouts and limits
AG-9	Easy to add sources, methods and AI applications	7	Source Connectors (C-08) share one interface; Persistence Repository (C-14) hides the database

3. Architecture Overview

3.1 Architectural Style

The system uses a layered architecture with a pipeline in the domain layer. It is deployed as one modular application (a "modular monolith") that exposes a REST API. A single deployable unit is the right size for a student mini-project, while the module boundaries keep each step replaceable later.

3.2 System Context

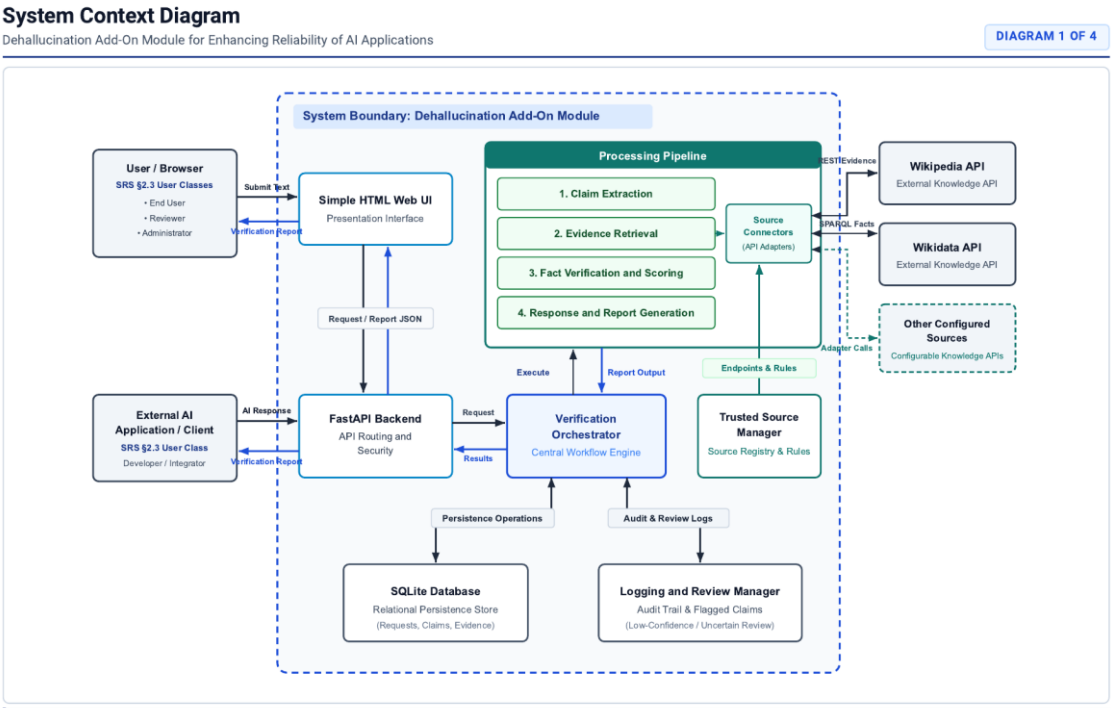


Figure 1 — System Context Diagram: Displays the Dehallucination Add-On Module system boundary, approved SRS §2.3 user classes (End User, Reviewer, Administrator, Developer/Integrator), internal orchestration and pipeline components, and external knowledge sources.

End Users, Reviewers and Administrators use the web UI in a browser. The Developer/Integrator connects an external AI application through the REST API. The module reads evidence from Wikipedia, Wikidata and other configured sources, and keeps its records in the SQLite database. All four user classes are those defined in SRS Section 2.3.

3.3 Layers

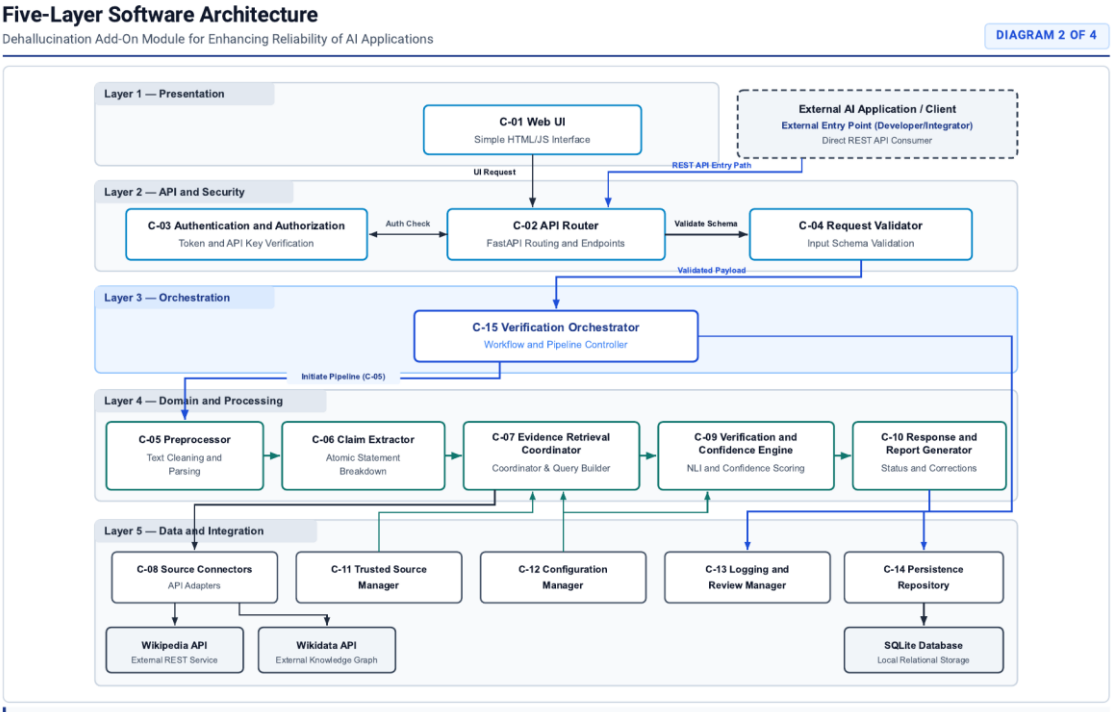


Figure 2 — Five-Layer Software Architecture: Displays components C-01 through C-15 across the five mandatory layers, showing the presentation web UI and external API entry point, C-03 authentication, C-02 router, C-04 schema validator, C-15 orchestrator, and sequential domain processing.

Layer	Components	Responsibility	Main technology
1. Presentation	C-01; external AI application	Lets users and AI applications submit text and read results	HTML + JavaScript page; API clients
2. API and Security	C-02, C-03, C-04	Receives HTTP requests, authenticates the caller, checks the role, validates the request	FastAPI, Pydantic, token and API-key authentication
3. Orchestration	C-15	Runs the verification steps in order, handles retries, failures and timeouts, assembles the final result	Python service class
4. Domain and Processing	C-05, C-06, C-07, C-09, C-10	Preprocessing, claim extraction, evidence retrieval coordination, NLI verification with confidence scoring, response and report generation	spaCy, Hugging Face Transformers (NLI model)
5. Data and Integration	C-08, C-11, C-12, C-13, C-14	Source connectors, source registry, configuration, logging and review, persistence	Wikipedia and Wikidata APIs, SQLite, YAML/.env

Rule: dependencies point downward only. A component may use components in its own layer or in lower layers through defined interfaces, and never a component in a higher layer. C-15 coordinates the processing workflow and manages the interactions between components. Domain components do not depend on the API layer, so they can be unit-tested without starting a web server. Access to persistence and to source-management functions is provided through defined interfaces (C-14 and C-11).

4. Component View

4.1 Component Catalogue

The 15 components below are logical modules inside one application, not separate services. Their IDs are the same in all diagrams and in the Jira Backlog and Test Plan.

ID	Component	Responsibility	Input → Output	SRS requirements
C-01	Web UI	Simple HTML/JS pages: submit text, view claim statuses, confidence and evidence; admin and review pages	User actions → API calls and rendered report	FR-23 to FR-25; 3.1
C-02	API Router	FastAPI routing and endpoints; returns structured JSON results and error codes	HTTP request → JSON response	FR-26 to FR-28
C-03	Authentication and Authorization	Verifies tokens (web users) and API keys (clients); enforces role-based permissions; authentication is required for every request, including the local demo	Credentials + role → allow/deny	FR-30; 6.3; 6.5
C-04	Request Validator	Checks that the text is present, well-formed and in the supported format; rejects invalid input with an error	Raw request → validated payload or error	FR-02, FR-03
C-05	Preprocessor	Cleans and normalises the text (whitespace, symbols) while keeping the original and a mapping between positions in the processed text and in the original text, so claim offsets stay accurate	Validated text → clean text + position mapping	FR-04
C-06	Claim Extractor	spaCy sentence splitting plus a claim filter; keeps factual statements and drops opinions, questions and greetings; records the start and end offset of each claim, converted to positions in the original text through the C-05 mapping	Clean text → list of claims (IDs, start/end offsets) or "no verifiable claim"	FR-05 to FR-07
C-07	Evidence Retrieval Coordinator	Builds a query per claim, requests evidence through C-08 from the enabled sources, applies bounded retries and a cache, records failures	Claim → evidence items or failure	FR-08 to FR-11
C-08	Source Connectors	One adapter per source type (Wikipedia, Wikidata, generic API, mock) with the same interface; returns snippets and references	Query → snippets + source reference	FR-09, FR-10; 3.2
C-09	Verification and Confidence Engine	Applies the evidence quality rules and the NLI model, assigns Supported / Contradicted / Uncertain, computes the confidence estimate and sets the review flag	Claim + evidence → status, confidence, review flag	FR-12 to FR-18
C-10	Response and Report Generator	Builds claim-level and overall results, marks contradicted and uncertain content, attaches the evidence-supported suggested statement beside the original text	Claim results + original text → report	FR-19 to FR-25
C-11	Trusted Source Manager	Add, edit, enable, disable and remove trusted sources; validates the configuration; administrators only	Admin request → stored source configuration	FR-29, FR-30; 6.5
C-12	Configuration Manager	Loads thresholds, timeouts, retry limit, claim limit and logging switch from a config file	Config file → settings	FR-18; 6.1; 7
C-13	Logging and Review Manager	Writes the audit log (events, errors), and falls back to the application log file when the database is unavailable; lists flagged and uncertain results for reviewers; masks sensitive data	Events → audit records; query → records	SRS 5.10; 6.3
C-14	Persistence Repository	All database reads and writes behind one interface	Objects ↔ SQLite rows	Section 7; Appendix B
C-15	Verification Orchestrator	Runs the pipeline (C-05 to C-10) in order, retries retrieval within a limit, turns failures into Uncertain or incomplete results, stores results and calls logging	Validated request → final verification result	All of 5.1 to 5.7

4.2 Notes on the Component Design

- C-09 combines verification and confidence scoring in one component, as in the approved diagrams. Inside it, the evidence-quality check, the NLI comparison and the confidence calculation are separate functions, so each can be tested alone (SRS 6.4).

- C-15 controls the order of the verification workflow. Components perform their assigned tasks through defined interfaces, while C-15 coordinates the interactions between the retrieval, source-management, verification and persistence components.
- C-03 sits beside C-02 as a security check and is not a step in the processing path.

5. Behaviour and Processing Rules

5.1 Runtime Flow

The flow is shown in Figure 3 and follows the SRS sequence in Figure 3 of the SRS.

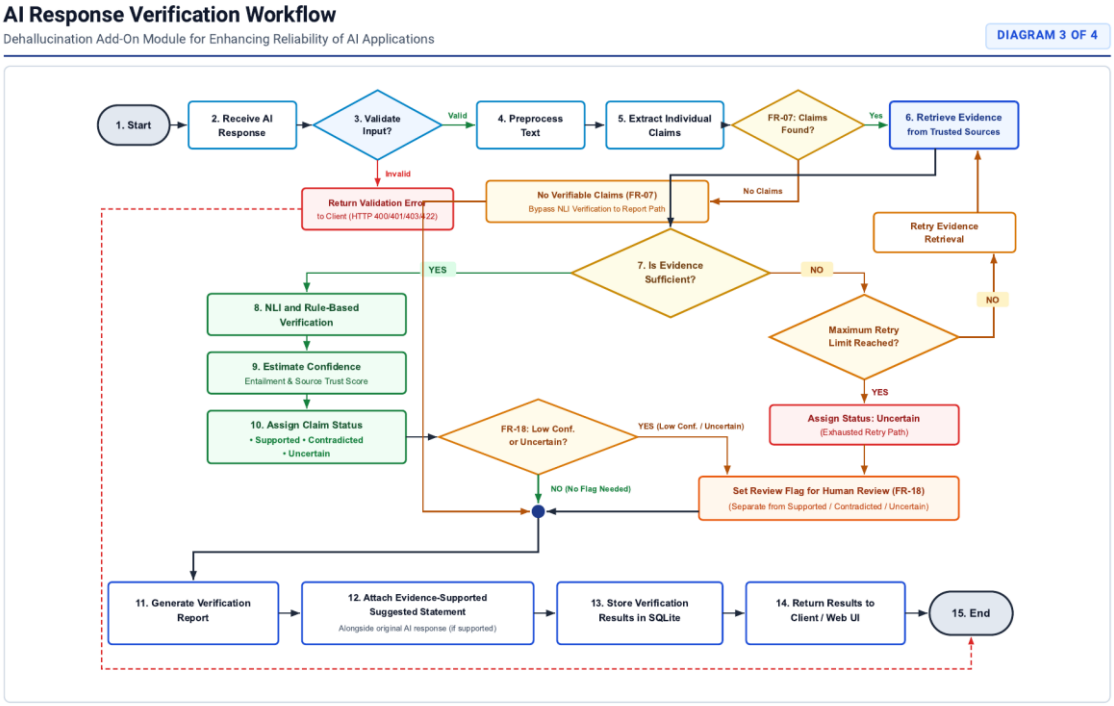


Figure 3 — AI Response Verification Workflow: Traces input validation with error exits, claim extraction with FR-07 no-verifiable-claims routing, bounded retry evidence retrieval, NLI verification, separate FR-18 review flagging for low-confidence or uncertain claims, and evidence-supported suggested statement attachment.

1. The user submits the generated text through the web UI (C-01), or an external AI application sends it through the REST API (C-02).
2. C-03 verifies the authentication token or API key and checks the caller's role and permissions. If authentication fails, the system returns HTTP 401; if the caller lacks permission, HTTP 403.
3. C-04 validates the request. A request with an invalid format or missing required fields returns HTTP 400; a correctly formatted request with invalid or unsupported input returns HTTP 422. Processing stops if validation fails.
4. C-05 preprocesses the text and keeps the original together with a position mapping.
5. C-06 extracts the claims. If there are none, a "no verifiable claims" report is returned and verification is skipped (FR-07).
6. For each claim, C-15 calls C-07, which gets the enabled sources from C-11 and queries them through C-08, in parallel, with a timeout per source.
7. If the evidence is insufficient, C-07 retries within the retry limit (proposed: 1 retry). When the limit is reached the claim gets the status Uncertain.
8. C-09 applies the evidence quality check and the NLI model, then estimates the confidence and assigns the status.
9. If the confidence is low or the status is Uncertain, the review flag is set (FR-18), following the rule in Section 5.2. The flag is separate from the status. A Contradicted claim with high confidence is marked in the response but not review-flagged.

10. C-10 builds the verification report and attaches the evidence-supported suggested statement beside the original response.
11. C-15 stores the results through C-14, and C-13 writes the audit log. If the database is unavailable, nothing can be stored and the audit log cannot be written either: the completed result is still returned with "stored": false, and C-13 writes the storage failure and the event to the application log file as a fallback. The response never implies that a result was stored when it was not.
12. The result is returned to the client or shown in the web UI.

5.2 Claim Status Rules

The NLI model receives the retrieved snippet as the premise and the claim as the hypothesis, and returns three probabilities: entailment (E), contradiction (C) and neutral (N). For one claim with several snippets, let E_max and C_max be the highest entailment and contradiction values across the qualifying snippets. The threshold values below are proposed design parameters (see Section 5.5) and will be tuned on the labelled claim set used in the Test Plan.

Condition	Status	Confidence shown
No evidence found, or all sources failed or timed out after the retries	Uncertain (reason: no evidence / retrieval failure)	0 (not applicable)
E_max ≥ 0.70 and C_max < 0.30	Supported	E_max
C_max ≥ 0.70 and E_max < 0.30	Contradicted	C_max
Anything else (neutral result, weak scores, or evidence that conflicts)	Uncertain	Highest class score (E_max, C_max or N)

When no qualifying evidence is available, confidence is set to 0 as a not-applicable value because no usable NLI result is available for the claim. For claims with qualifying snippets, N represents the neutral score for an individual snippet, and the highest neutral score across those snippets is used when determining the confidence for an Uncertain status.

- **Evidence quality check (rules part of the hybrid method):** before the NLI step, only snippets from enabled trusted sources whose relevance to the claim reaches a minimum value are kept. If none qualify, the claim is Uncertain. For claims containing numbers, dates or named entities, those values must also appear in the qualifying snippet.
- **NLI is not proof:** a high NLI score alone never makes a claim Supported. Supported needs both a strong NLI result and qualifying evidence from an enabled trusted source, so a high score on weak, off-topic or low-quality evidence is not enough.
- **Review flag rule (used in the text, Figure 3 and the runtime flow):** review_flag is set to true when the status is Uncertain (including the no-evidence and exhausted-retry cases) or when the confidence is below the review threshold, whatever the status. A Contradicted claim with confidence at or above the threshold is not review-flagged, but it is still marked in the response for the user (FR-19, FR-20). The flag is an additional field. It is not a fourth status; the only official claim statuses are Supported, Contradicted and Uncertain.
- The confidence value is the NLI model's highest class score for the label that decided the status, and it is treated as an estimate. It shows how sure the model is about its own label; it is not the probability that the claim is factually true. For an Uncertain claim it can be high when the model is sure the evidence is neutral, so the API and the UI always show it next to the status.
- When a claim is Uncertain because no evidence could be retrieved, the result states the reason, so the user can see that verification could not be completed for that claim (SRS 6.2).

5.3 Overall Response Status

These are design labels for the whole response, stored as overall_status. They are not claim statuses and are kept separate from the three official claim statuses.

Overall status	Meaning
Verified	Every extracted claim is Supported
Partially Verified / Flagged	At least one claim is Contradicted or Uncertain
No Verifiable Claims	No factual claim found in the text. If claim extraction finds no verifiable claims, the overall status is No Verifiable Claims and the verification process (retrieval and NLI) is skipped.
Verification Incomplete	A system or service failure (for example the NLI model is unavailable or an internal error stops processing) prevents the verification process from completing; the results completed so far are returned, or HTTP 503 if there are none. Claims that were not reached are not listed in claims; their number is returned in unprocessed_claim_count, and they are never shown as verified. A claim that only lacks sufficient evidence is marked Uncertain instead. A response affected by an incomplete process is never shown as Verified. A failure only to store the result does not make verification incomplete (Section 5.1, step 11).

5.4 Response Processing Rule

The "configured project rule" of SRS 5.6 is set as: flag and qualify, never silently delete. A contradicted sentence is marked visibly and the supporting evidence is attached (this mark is separate from the review flag); an uncertain sentence is marked as unverified. Supported sentences stay unchanged. The original response is always kept.

The suggested statement is the evidence sentence from the trusted source that addresses the claim, shown beside the original text. It is taken from the retrieved evidence, so no text-generation model is needed and nothing new is invented. The final response returned to the client is the original text together with these marks and suggested statements, assembled when the result is returned.

5.5 Proposed Design Parameters

The SRS does not fix the values below. They are proposed implementation choices that need agreement with the project guide. They are not SRS requirements, and they are stored in the config file (C-12) so they can change without code changes.

Parameter	Proposed value	Status
Response-time target	95% of requests with up to 5 claims within 10 seconds	Proposed; SRS 6.1 says TBD
Supported rule	E_max ≥ 0.70 and C_max < 0.30	Proposed; tune on labelled claims
Contradicted rule	C_max ≥ 0.70 and E_max < 0.30	Proposed; tune on labelled claims
Review-flag confidence threshold	0.6	Proposed; FR-18 says "if a threshold is used"
Minimum snippet relevance	0.5	Proposed; tune on labelled claims
Per-source timeout	5 seconds	Proposed
Maximum retrieval retries	1 retry per claim	Proposed
Maximum claims per request	20	Proposed
Overall result names	Verified, Partially Verified/Flagged, No Verifiable Claims, Verification Incomplete	Design labels (Section 5.3)

6. Data Design

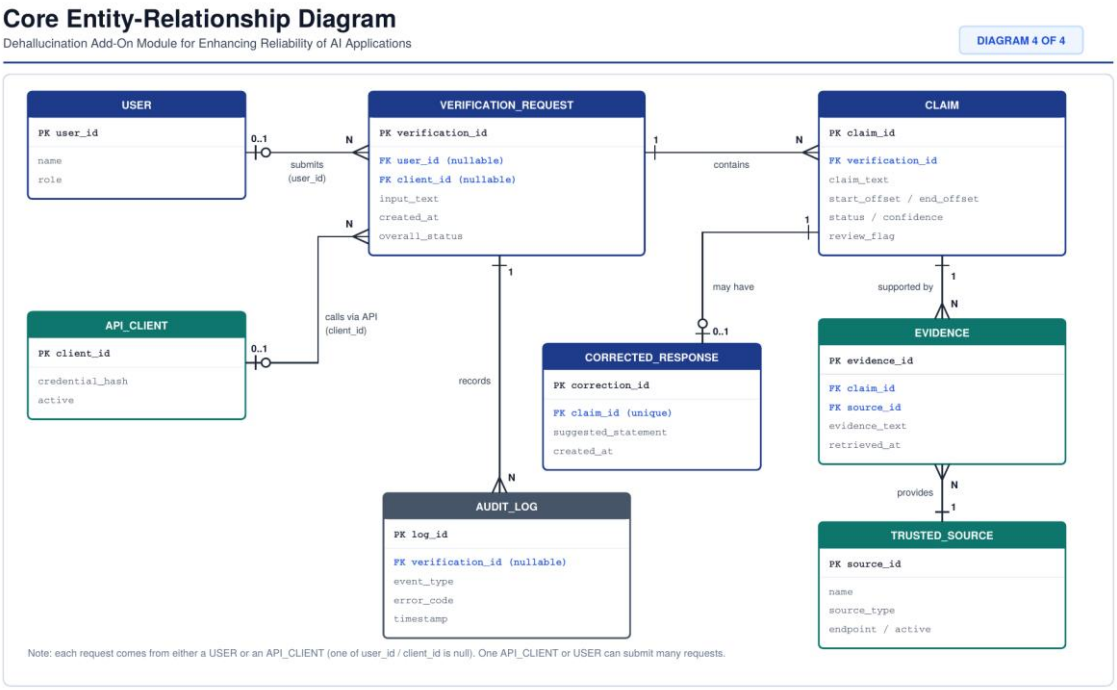


Figure 4 — Core Entity-Relationship Diagram: Displays 8 entities (USER, API_CLIENT, VERIFICATION_REQUEST, CLAIM, EVIDENCE, TRUSTED_SOURCE, CORRECTED_RESPONSE, AUDIT_LOG) with primary and foreign keys, nullable user_id and client_id on VERIFICATION_REQUEST (a request comes from either a USER or an API_CLIENT), errors recorded in AUDIT_LOG, and crow's-foot cardinalities.

The data model has eight entities. In CLAIM, start_offset and end_offset give the position of the claim in the original response (zero-based character positions; start included, end excluded), so the correct sentence can be found and marked. Because C-05 may change whitespace and symbols, the offsets are mapped back to the original text through the position mapping kept by C-05. CORRECTED_RESPONSE is linked to a claim through claim_id; the request is reached through that claim. API_CLIENT and AUDIT_LOG are additions to the entities described in the SRS: they are needed for API-key authentication (SRS 6.3) and for logging and review (SRS 5.10 and Section 7). Field lengths marked TBD in the SRS are fixed at implementation time. The Persistence Repository (C-14) lets the team move from SQLite to PostgreSQL later without changing the other components.

Entity	Purpose	Key fields
USER	Web users and their role (End User, Reviewer, Administrator)	user_id, name, role
API_CLIENT	External AI applications of Developer/Integrators; credentials are stored as hashes	client_id, credential_hash, active
VERIFICATION_REQUEST	One row per request; the caller is either a USER or an API_CLIENT, so one of user_id and client_id is null	verification_id, user_id, client_id, input_text, created_at, overall_status
CLAIM	Claims with their position in the original text, status, confidence and review flag	claim_id, verification_id, claim_text, start_offset, end_offset, status, confidence, review_flag
EVIDENCE	Evidence used for each claim, with its source	evidence_id, claim_id, source_id, evidence_text, retrieved_at

Entity	Purpose	Key fields
TRUSTED_SOURCE	Configured trusted sources and whether they are active	source_id, name, source_type, endpoint, active
CORRECTED_RESPONSE	Optional evidence-supported suggested statement for a claim (at most one per claim)	correction_id, claim_id, suggested_statement, created_at
AUDIT_LOG	Activity and error records for review and debugging	log_id, verification_id, event_type, error_code, timestamp

6.1 Mapping to SRS Appendix B

SRS Appendix B field	Stored in
Request ID	VERIFICATION_REQUEST.verification_id (called request_id in the API)
Generated Response	VERIFICATION_REQUEST.input_text
Claim ID, Claim Text	CLAIM.claim_id, CLAIM.claim_text
Verification Status, Confidence	CLAIM.status, CLAIM.confidence
Evidence, Source	EVIDENCE.evidence_text (snippet with its page reference) and the TRUSTED_SOURCE linked by source_id
Timestamp	VERIFICATION_REQUEST.created_at
User/Client ID	VERIFICATION_REQUEST.user_id or client_id
Error Code	AUDIT_LOG.error_code

6.2 Integrity Constraints

These constraints are part of the relational schema to be created in the implementation.

Table	Constraint
VERIFICATION_REQUEST	verification_id is the primary key, so every ID is unique, whether supplied by the client or generated by the server; a duplicate supplied ID is rejected (HTTP 422). user_id references USER and client_id references API_CLIENT; exactly one of them is set, because every accepted request is authenticated.
CLAIM	verification_id is required and references VERIFICATION_REQUEST. status is limited to Supported, Contradicted and Uncertain. confidence is between 0 and 1, inclusive. review_flag is true or false. start_offset is at least 0 and less than end_offset.
EVIDENCE	claim_id is required and references CLAIM; source_id is required and references TRUSTED_SOURCE.
CORRECTED_RESPONSE	claim_id is required, references CLAIM(claim_id), and is UNIQUE, so a claim has at most one suggested statement.
AUDIT_LOG	verification_id is optional and references VERIFICATION_REQUEST when set; it is null for events that do not belong to one request.
TRUSTED_SOURCE	A source referenced by any EVIDENCE row is never physically deleted. "Remove" then sets active to false and keeps the record, so historical evidence stays valid. A source that was never used may be deleted.
USER, API_CLIENT	Only password or key hashes are stored, never plain text. USER.role is limited to End User, Reviewer and Administrator.

7. API Design

Method and path	Purpose	Allowed roles	FR
POST /api/v1/verify	Verify a generated response	End User, Developer/Integrator	FR-01, FR-26 to FR-28

Method and path	Purpose	Allowed roles	FR
GET /api/v1/results/{request_id}	Fetch a stored result	Requesting user/client (own results); Reviewer (own results and results with a flagged claim); Administrator (any result)	FR-23 to FR-25
GET /api/v1/results	List the results the caller may fetch (same rules as above)	Authenticated users	FR-23, SRS 5.10
POST /api/v1/auth/login	Log in and receive a token	Registered web users	6.3
GET /api/v1/sources	List trusted sources	Administrator	FR-29
POST /api/v1/sources	Add a trusted source	Administrator	FR-29, FR-30
PUT /api/v1/sources/{source_id}	Update a source, or enable or disable it	Administrator	FR-29, FR-30
DELETE /api/v1/sources/{source_id}	Remove a source (a source already used as evidence is only disabled, see Section 6.2)	Administrator	FR-29, FR-30
GET /api/v1/review/flagged	List flagged and uncertain results	Reviewer, Administrator	SRS 5.10
GET /api/v1/logs	View the audit log	Administrator	SRS 5.10
GET /health	Service health check (no sensitive details)	Public	6.1, 6.4

7.1 Sample Request

```
POST /api/v1/verify
{ "request_id": "REQ-100", "response": "The Eiffel Tower is in Paris." }
```

request_id is optional. SRS 3.3 lists a request identifier in the request, so a client may supply one. If it is omitted, the server generates a unique ID. A duplicate ID is rejected with HTTP 422. The accepted or generated ID is returned in the response and is used to fetch the stored result.

7.2 Sample Response

The field structure below is complete. Only the evidence snippet text and URL are shortened with "...". Long lines wrap in this document only; the JSON itself is valid.

```
{ "request_id": "REQ-100", "overall_status": "Partially Verified/Flagged",
  "original_response": "The Eiffel Tower is in Paris. It was built in 1950.",
  "claims": [
    { "claim_id": "C1", "claim_text": "The Eiffel Tower is in Paris.",
      "start_offset": 0, "end_offset": 29, "status": "Supported",
      "confidence": 0.93, "review_flag": false,
      "evidence": [ { "snippet": "...", "source": "Wikipedia", "url": "..." } ],
      "suggested_statement": null },
    { "claim_id": "C2", "claim_text": "It was built in 1950.",
      "start_offset": 30, "end_offset": 51, "status": "Contradicted",
      "confidence": 0.88, "review_flag": false,
      "evidence": [ { "snippet": "...", "source": "Wikipedia", "url": "..." } ],
      "suggested_statement": "The Eiffel Tower was completed in 1889." } ],
  "marked_response": "The Eiffel Tower is in Paris. It was built in 1950. [Contradicted]",
  "final_response": "The Eiffel Tower is in Paris. It was built in 1950. [Contradicted] Suggested:
The Eiffel Tower was completed in 1889.",
  "unprocessed_claim_count": 0, "stored": true, "timestamp": "2026-10-24T10:30:00Z" }
```

- **original_response:** the submitted text, never changed.
- **start_offset, end_offset:** zero-based character positions of the claim in original_response (start included, end excluded).
- **suggested_statement:** the evidence-supported wording taken from the retrieved evidence; null when the claim is Supported or no suitable evidence sentence exists.
- **marked_response:** the original text with a status mark after each Contradicted or Uncertain claim.

- **final_response:** the response returned for display: the marked text with each suggested statement attached beside its mark.
- **confidence:** the NLI model's highest class score (0 to 1) for the label that decided the status. It is not the probability that the claim is true; always read it together with status.
- **unprocessed_claim_count:** the number of extracted claims that were not verified because processing stopped. It is 0 for a complete result; when it is above 0 the overall_status is Verification Incomplete.
- **stored:** true when the result was saved. False when saving failed: the result is still returned, but it cannot be fetched again later.
- In this example C2 is Contradicted with confidence 0.88, so it is marked in the response but, under the rule in Section 5.2, review_flag stays false.

Error responses use standard HTTP codes (400 invalid format or missing fields, 401 authentication failed, 403 not permitted, 404 unknown or inaccessible request ID, 422 invalid or unsupported input, including a duplicate request ID, 429 rate limit if implemented, 503 service unavailable) with a body containing an error code and message, as required by FR-28. Endpoint paths and schemas are the structure for implementation and may be refined in detail without changing the architecture.

7.3 Role Permissions

The four roles are the user classes defined in SRS Section 2.3: End User, Developer/Integrator, Administrator and Reviewer. A Reviewer may fetch own results and results that contain at least one review-flagged claim, which keeps Reviewer access limited to the records needed for review (SRS 6.5). An Administrator may fetch any result. End Users, Reviewers and Administrators have USER accounts; the Developer/Integrator is represented by an API_CLIENT with an API key.

Function	End User	Developer / Integrator	Reviewer	Administrator
Submit and view own results	Yes	Yes (API key)	Yes	Yes
View flagged / uncertain results	No	No	Yes	Yes
View other users' results	No	No	Only results with a flagged claim	Any result
View audit log	No	No	Limited	Yes
Manage trusted sources	No	No	No	Yes

8. Technology Stack

Component	Proposed technology	Reason
Backend and API (C-02, C-15)	Python + FastAPI (served by Uvicorn)	SRS 2.4 expects a Python backend; built-in JSON validation, automatic API documentation, easy to test
Claim extraction (C-06)	spaCy sentence splitting + claim filter (rules)	Runs locally with no API key or cost; repeatable results; separable from other components
Evidence retrieval (C-07, C-08)	Wikipedia and Wikidata APIs + configurable trusted sources	Free, structured, and fits the "knowledge graph" idea in the problem statement; extra sources added through the admin list
Fact verification (C-09)	NLI model (pretrained, via Hugging Face Transformers) + rules	Its three outputs map to Supported, Contradicted and Uncertain (FR-13); the exact model is chosen in implementation
Confidence scoring (C-09)	NLI model score with evidence-aware rules, treated as an estimate	Matches SRS 2.6; simple and explainable
Database (C-14)	SQLite initially	No server setup; relational as allowed by SRS 2.4; replaceable through the repository

Component	Proposed technology	Reason
User interface (C-01)	Simple HTML web page (+ JavaScript calls to the API)	Meets SRS 3.1 with little effort; served by FastAPI as static files
Testing tools	pytest + Postman	Unit/integration tests in Python; API testing and demos
Other	Git, Jira, YAML/.env configuration	Version control, tracking, and configuration kept outside core logic (SRS 7)

9. Design Decisions

ID	Decision	Alternatives considered	Reason and trade-off
DD-01	Layered modular monolith	Microservices	Simpler to build, test and deploy for a mini-project; module boundaries allow later splitting
DD-02	Synchronous REST endpoint first; retrieval per claim runs in parallel threads	Asynchronous job queue	Suits interactive chatbot use (SRS 6.1); request IDs allow asynchronous processing to be added later if needed
DD-03	spaCy + rules for claim extraction	LLM-based extraction	Local, free, repeatable; trade-off: lower recall on complex sentences
DD-04	Hybrid verification: evidence-quality rules + NLI model	Keyword matching; LLM as judge	Explicit support/contradiction/neutral labels with scores, and rules guard against weak evidence; trade-off: model accuracy limits
DD-05	Neutral, missing or conflicting evidence maps to Uncertain	Treat "no contradiction" as supported	Required by SRS 2.5 and FR-15; reduces false "verified" results at the cost of more Uncertain outputs
DD-06	Three official statuses plus a separate review flag	A fourth status such as "Review Required"	Matches SRS FR-13 and Appendix B exactly; the flag adds attention without changing the status set
DD-07	Flag and qualify bad content; never silently delete	Remove or rewrite content	Preserves uncertainty (SRS 6.2)
DD-08	Sources stored in the database and managed by admin	Hard-coded source list	Required by SRS 6.5 and FR-29; trade-off: needs a source-management feature
DD-09	One connector interface per source type	Direct API calls in the retriever	New sources can be added without changing C-07 (SRS 7)
DD-10	SQLite behind a Persistence Repository	Direct SQL everywhere; PostgreSQL now	Fast setup now; storage can be replaced later
DD-11	Role-based access: API keys for integrators, token login for web users	No authentication in the first version	Needed for FR-30 and SRS 6.3; password and key hashes are stored, never plain text
DD-12	Thresholds and limits in a config file	Constants in code	Allows tuning without code changes (FR-18, SRS 7)
DD-13	Models loaded once at start-up; evidence cache with expiry	Load per request	Avoids slow requests and repeated retrieval (SRS 6.1)
DD-14	Bounded retries and explicit failures	Unlimited retries; silent failure	Handles temporary source problems; a failed retrieval is always reported (FR-11) and never shown as verified
DD-15	The suggested statement is extracted from evidence and shown beside the original	Generate corrected text with a language model	Meets FR-21 without a text-generation model, which could add new errors (SRS 6.2)
DD-16	Caller identity stored as either user_id or client_id	One combined account table	Web users and API clients authenticate differently; both are traceable to each request

10. Quality Attribute Tactics

Attribute (SRS 6.x)	Tactic in the architecture
Performance (6.1)	Parallel retrieval per claim; 5-second proposed timeout per source; bounded retries; evidence cache; models loaded at start-up; a maximum number of claims per request (proposed 20); all values are proposed parameters, see Section 5.5
Safety (6.2)	Fail-safe status rules (Section 5.2); Uncertain status (with a stated reason) for claims without evidence; Verification Incomplete when the process itself fails; visible unverified markers in the final response
Security (6.3)	Authentication and RBAC (C-03); hashed passwords and API keys; HTTPS for deployment; masked or minimal logging of user text; admin endpoints isolated
Reliability (6.4)	Per-source error handling; failures are recorded and never reported as success
Maintainability and testability (6.4)	Layered design; separate components with defined interfaces; the domain layer has no web dependencies
Interoperability (6.4)	REST + JSON API usable by any AI application
Traceability (6.4)	Request ID, claim ID and evidence ID link every result to its claims and sources in the database

11. Deployment View

The whole module runs as one Python process on a Windows or Linux machine, started with Uvicorn with a single worker, so the models and the evidence cache are loaded once and shared. Running several workers would load the models in each and would not share the cache, so it is outside this design. The HTML pages are served by the same FastAPI application. The SQLite file and the config file sit next to the application, and the spaCy and NLI models are loaded into memory at start-up. Outbound HTTPS calls go to the Wikipedia and Wikidata APIs and any other configured source. Browsers and external AI applications connect over HTTPS (HTTP is allowed only in a local test setup). For the demo, one laptop can act as both client and server.

12. Proposed Code Structure

```
dehallucination/
  app/
    main.py                    # FastAPI start-up
    api/router.py              # C-02 routes and schemas
    api/auth.py                # C-03
    api/validator.py           # C-04
    services/orchestrator.py   # C-15
    domain/
      preprocess.py            # C-05
      claim_extractor.py       # C-06
      retrieval.py             # C-07
      verifier.py              # C-09 (rules + NLI + confidence)
      report.py                # C-10
    sources/                   # C-08 connectors: wikipedia, wikidata, generic, mock
    admin/source_manager.py    # C-11
    core/config.py             # C-12
    core/logging_review.py     # C-13
    db/                        # C-14 models + repository
  web/                         # C-01 HTML, JS, CSS
  config.yaml
  tests/                       # unit, integration, api
```

13. Requirements to Components Traceability

The ranges follow Appendix C of SRS v1.0. The component links are proposed architectural links, not claims of implementation.

Requirements	System feature	Components
FR-01 to FR-03	5.1 Response Input and Validation	C-02, C-03, C-04

Requirements	System feature	Components
FR-04	5.1 Response Input and Validation	C-05
FR-05 to FR-07	5.2 Claim Extraction	C-06, C-15
FR-08 to FR-11	5.3 Trusted Evidence Retrieval	C-07, C-08, C-11, C-12
FR-12 to FR-15	5.4 Fact Verification	C-09, C-15
FR-16 to FR-18	5.5 Confidence Scoring	C-09, C-12
FR-19 to FR-22	5.6 Hallucination Detection and Response Processing	C-09, C-10, C-14
FR-23 to FR-25	5.7 Evidence and Result Presentation	C-01, C-02, C-10
FR-26 to FR-28	5.8 Verification API	C-02, C-03, C-04, C-15
FR-29, FR-30	5.9 Trusted Source Management	C-03, C-11, C-14
SRS 5.10	5.10 Logging and Review (no FR range in Appendix C)	C-13, C-14, C-03
NFR 6.1 to 6.5	Nonfunctional requirements	C-15, C-07, C-03, C-12

14. Risks, Limitations and Open Items

- Coverage: Wikipedia and Wikidata do not cover every claim, so many claims may be Uncertain. This is allowed by the SRS and is safer than guessing.
- NLI accuracy: the model can make mistakes on long, numeric or multi-fact claims; thresholds are tuned on the labelled claim set and the confidence value is treated as an estimate.
- Start-up time and memory: the NLI model is loaded once at start-up; a smaller model may be used on low-memory machines.

14.1 Items to agree with the project guide

- The proposed design parameters in Section 5.5 (they are not SRS requirements).
- A short source-reliability rule for choosing trusted sources (SRS 2.6).
- Section 5.10 of the SRS has no FR numbers; this document traces it by section.
- API_CLIENT and AUDIT_LOG are design additions to the SRS data items.
- The suggested statement is taken from evidence and no generative model is used; confirm that this satisfies FR-21.

14.2 Items finalised during implementation

- The exact NLI model and the exact field types and lengths that are TBD in SRS Appendix B.
- Final request and response schemas and endpoint details, within the structure given in Section 7.

15. Approval Status

Status: final design for Phase 1, based on SRS v1.0, the technology stack approved by the team and the approved set of four diagrams. It describes the intended architecture only; implementation, integration and testing have not been done.